

TREE 2

PROBLEM 7

QUESTION:

DS	Session	Tree 2	Question Information	Level 1 Challenge 77
Problem Description: Any sequence A of size n is called B-sequence if: $A_1 < A_2 < \dots < A_k > A_{k+1} > A_{k+2} > \dots > A_n \text{ where } 1 \leq k \leq n.$ That is, a sequence which is initially strictly increasing and then strictly decreasing (the decreasing part may or may not be there). All elements in A except the maximum element comes atmost twice (once in increasing part and once in decreasing part) and maximum element comes exactly once. All elements coming in decreasing part of sequence should have come once in the increasing part of sequence. You are given a B-sequence S and Q operations. For each operation, you are given a value val. You have to insert val in S if and only if after insertion, S still remains a B-sequence . After each operation, print the size of S. After all the operations, print the sequence S. Hint: Think of using some data structure to support insertion of elements in complexity better than linear. Input Constraints: $1 \leq N \leq 10^5$ $1 \leq S_i \leq 10^9$ $1 \leq Q \leq 10^5$ $1 \leq val \leq 10^9$ Given sequence S is a B-sequence, Input Format: First line consists of an integer N, denoting size of S. Second line consists of N space separated integers, denoting elements of S. Next line consists of an integer Q, denoting number of operations. Each of the following Q lines consists of an integer val. Output Format: After each operation, print the size of S in a new line. After all operations, print the sequence S.				
Logical Test Cases Test Case 1 INPUT (STDIN) <pre>4 1 2 5 2 4 5 1 3 2</pre> EXPECTED OUTPUT <pre>4 5 6 6 1 2 3 5 2 1</pre> Test Case 2 INPUT (STDIN) <pre>5 3 4 5 1 3 3 6 2 3 1</pre> EXPECTED OUTPUT <pre>6 7 7 1 2 3 4 5 6 3</pre>				
Mandatory Test Cases Test Case 1 KEYWORD <pre>for(i=0;i<N;i++)</pre>				

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

39

SOURCE CODE:

```
#include <stdio.h>

#define MAX_VAL_BOUND 1000005

int element_occ[MAX_VAL_BOUND];

int main() {
    int initial_len;
    if (scanf("%d", &initial_len) != 1) return 0;

    int peak_value = 0;
    for (int i = 0; i < initial_len; i++) {
        int x;
        scanf("%d", &x);
        element_occ[x]++;
        if (x > peak_value) peak_value = x;
    }

    int total_elements = initial_len;
    int insertions;
    if (scanf("%d", &insertions) != 1) return 0;

    while (insertions--) {
        int item;
        scanf("%d", &item);
        if (item > peak_value) {
            element_occ[item] = 1;
            peak_value = item;
            total_elements++;
        } else if (item < peak_value) {
            if (element_occ[item] < 2) {
                element_occ[item]++;
                total_elements++;
            }
        }
        printf("%d\n", total_elements);
    }

    int first_flag = 1;
    for (int v = 1; v <= peak_value; v++) {
        if (element_occ[v] >= 1) {
            if (!first_flag) printf(" ");
            printf("%d", v);
            first_flag = 0;
        }
    }
    for (int v = peak_value - 1; v >= 1; v--) {
        if (element_occ[v] == 2) {
            printf(" %d", v);
        }
    }
    printf("\n");
    return 0;
}
```

OUTPUT:

```
5
1 2 5 4 3
2
2
5
6
6
1 2 2 5 4 3
```

```
5
3 4 5 1 3
6
2
3
1
6
7
```